

Synchronization（同步机制）的大题通常是整张试卷中最灵活、最容易让人毫无头绪的部分。但只要你掌握了它的**核心套路（Design Patterns）**，其实这些题目就像是拼图游戏。

以下为你总结的关于 Synchronization 大题的**万能解题模板、常见题型及历年真题映射**，帮助你在考试时迅速理清思路。

💡 核心解题心法 (The Golden Rules)

做任何 Synchronization 填空或设计题前，先在脑海中明确以下三个概念：

- Mutex (互斥锁)**：只用来**保护共享变量** (Shared Variables)。通常初始值为 **1**。套路一定是 `wait(mutex) -> 修改变量(如 count++) -> signal(mutex)`。
- Condition / Signaling Semaphore (条件/同步信号量)**：用来**控制执行顺序** (谁等谁)。通常初始值为 **0**。套路是：等待方调用 `wait(sync_sem)`，满足条件方调用 `signal(sync_sem)`。
- Monitor (管程)**：自带一把大锁，进入函数自动加锁，退出自动解锁。在 Monitor 中等待必须用 `while (condition) wait(cond);`，唤醒用 `signal(cond)`。

一、交替执行与顺序控制 (Alternating Sequence)

- 题目特征**：要求多个线程按特定顺序执行（如循环打印 P1, P0, P1... 或 "ACA", "ACB"）。
- 解题套路 (使用 POSIX Semaphores: `sem_init`, `sem_wait`, `sem_post`)**:
 - 谁先执行，谁的信号量初始值为 **1**，其余为 **0**。
 - 线程内部的结构永远是：`wait(自己的信号量) -> 执行动作 (Critical Section) -> signal(下一个线程的信号量)`。
- 历年真题**:
 - 2023 Fall Q5.1: 打印 P1, P0, P1... （初始值 `S0=0, S1=1`。P1执行完 `sem_post(&S0)`，P0执行完 `sem_post(&S1)`）。
 - 2024 Spring Q5.2: 按概率打印 ACA / ACB。利用数学概率推算初始值和后续跳转，本质也是信号量顺序控制。

二、经典同步问题变种 (Classic Synchronization Problems)

这类题目最长、最吓人（如过河问题、理发师问题、校车接送问题），但只要按以下三步走，填空非常简单。

1. Semaphore 实现版本 (以理发师 Barbershop 为例)

- 特征**：手动管理 `mutex`，以及各种实体状态（如 `customers`, `barber_busy`）。
- 解题步骤**:
 - Step 1: 找共享变量**。看到 `customers++` 或 `customers--`，前后必定需要 `P(&mutex)` 和 `V(&mutex)` 包裹。
 - Step 2: 找等待条件**。顾客要等理发师 (`P(&b_sem)`)，理发师要等顾客 (`P(&c_sem)`)。
 - Step 3: 找唤醒条件**。顾客来了，唤醒理发师 (`V(&c_sem)`)；理发师准备好了，唤醒顾客 (`V(&b_sem)`)。
- 历年真题**：2024 Fall Q5.2 (Barbershop 理发师与 VIP 顾客问题)。

2. Monitor 实现版本 (以校车 Bus Shuttle 为例)

- **特征：**函数外壳是 `Monitor`，内部使用 `condition` 变量。不需要手动写 `mutex`。
- **解题公式 (Template)：**

```
// 1. 等待条件满足 (必须用 while)
while (条件不满足, 比如车没满 count < 4) {
    waiting++;
    cond.wait(); // 等待
    waiting--;
}
// 2. 执行核心动作 (上车/下车)
// 3. 唤醒其他等待者
if (满足某条件, 比如车满了/全下车了) {
    // 唤醒多个使用 for 循环
    for(int i=0; i<N; i++) cond.signal();
}
```

- **历年真题：**
 - 2025 Fall Q6: Bus Shuttle (校车接送)。核心填空：用 `while` 检查条件，调用 `can_board[location].wait()`；车满时用 `for(i=0; i<4; i++) can_depart.signal()` 唤醒所有人。

三、读者-写者问题 (Readers-Writers Problem)

- **题目特征：**涉及 `read_count` 变量，`mutex` 信号量，`wrt_mutex` 信号量。
- **核心记忆点 (必须背熟的结构)：**
 - `mutex` 仅用于保护 `read_count` 的增减。
 - `wrt_mutex` 用于保护数据本身，被 **Writer** 以及“第一个/最后一个” **Reader** 争夺。
 - 第一个进来的 **Reader** 负责上锁：`if (readcnt == 1) wait(wrt_mutex);`
 - 最后一个离开的 **Reader** 负责解锁：`if (readcnt == 0) signal(wrt_mutex);`
- **历年真题：**2024 Spring Q5.1 (填空题：考察写者优先 Write-Priority 的变种。填空围绕 `mutex_1`, `mutex_2`, `r_count_mutex` 展开)。

四、同步屏障 (Synchronization Barrier)

- **题目特征：**N 个线程并发，要求所有人都到达某个点后，大家才能一起进入下一个阶段 (Stage-2)。
- **解题公式 (代码结构必须熟悉)：**

```
mutex.wait();
count++; // 记录到达的线程数
mutex.signal();

if (count == N) {
    barrier.signal(); // 最后一个到达的线程负责触发第一枪
}
barrier.wait(); // 所有人都卡在这里
barrier.signal(); // 级联唤醒 (Cascading wakeup): 被唤醒的人立刻唤醒下一个
```

- **历年真题:** 2023 Fall Q5.2。考察 Barrier 如果没有最后一句 `barrier.signal()` 会发生什么, 以及 `barrier` 变量可能的最终值 ($1 \leq barrier \leq N$)。

五、并发变量追踪 (Concurrency Tracing & Race Conditions)

- **题目特征:** 给出两段毫无锁保护的代码 (如 Thread 1 和 Thread 2), 里面对变量 `a`, `b`, `c` 进行读写, 问最终 `c` 的可能值。
- **极速破解法:**
 1. 把高级语言拆成机器指令思维: `c = b - a` 意味着 先读 `b`, 再读 `a`, 然后计算, 再写 `c`。
 2. **列举中断点 (Context Switch Points):** 假设 Thread 1 在读完 `b` 但还没读 `a` 时被中断, 此时 Thread 2 执行完了, 改变了 `a` 的值。再切回 Thread 1, 它用的就是**旧的 `b` 和新的 `a`**。
 3. 穷举几种关键的 Interleaving (交错) 情况计算结果。
- **历年真题:** 2024 Fall Q5.1。初始 $a = -10, b = 5, c = 9$ 。如果 Thread 2 在 Thread 1 赋值前后发生, 会产生 `-5`, `-4`, `-1`, `12`, `13` 等多种结果。

考场实战 Action Plan (看到题目怎么做)

1. **先看提供给你的变量/信号量是什么:**
 - 如果提供了 `sem_t S0, S1` -> 肯定是**顺序控制题**。找准谁先起步, 分配初始值。
 - 如果是 `Monitor` 结构 -> 肯定是**条件变量题**。找准 `.wait()` 填在哪里 (通常在 `while` 里), 找准 `.signal()` 填在哪里 (通常在事件完成后的 `if` 里)。
2. **分析临界区 (Critical Section):**
 - 如果代码里有 `count++`、`customers--` 这种加减操作, 它的正上方一定要填 `wait(mutex)`, 正下方一定要填 `signal(mutex)`。
3. **小心级联唤醒 (Cascading Wakeup):**
 - 如果要唤醒多个等待的线程, `Monitor` 用 `for (int i=0; i<N; ++i) cond.signal()`, `Semaphore` 用类似 `barrier.signal()` 链式唤醒或循环 `V(sem)`。